

1.1 Introdução

Um algoritmo é um processo sistemático para a resolução de um problema. O desenvolvimento de algoritmos é particularmente importante para problemas a serem solucionados em um computador, pela própria natureza do instrumento utilizado. Neste livro são abordados apenas esses problemas. Existem dois aspectos básicos no estudo de algoritmos: a *correção* e a *análise*. O primeiro consiste em verificar a exatidão do método empregado, o que é realizado através de uma prova matemática. A análise visa à obtenção de parâmetros que possam avaliar a eficiência do algoritmo em termos de tempo de execução e memória ocupada. A análise é realizada através de um estudo do comportamento do algoritmo. Ambos os aspectos serão considerados neste livro.

Um algoritmo computa uma *saída*, o resultado do problema, a partir de uma *entrada*, as informações inicialmente conhecidas e que permitem encontrar a solução do problema. Durante o processo de computação o algoritmo manipula *dados*, gerados a partir de sua entrada. Quando os dados são dispostos e manipulados de uma forma homogênea, constituem um *tipo abstrato de dados*. Este é composto por um modelo matemático acompanhado por um conjunto de operações definido sobre esse modelo. Um algoritmo é projetado em termos de tipos abstratos de dados. Para implementá-los numa linguagem de programação é necessário encontrar uma forma de representá-los nessa linguagem, utilizando tipos e operações suportadas pelo computador. Na representação do modelo matemático emprega-se uma *estrutura de dados*, o assunto deste livro.

As estruturas diferem umas das outras pela disposição ou manipulação de seus dados. A disposição dos dados em uma estrutura obedece a condições preestabelecidas e caracteriza a estrutura.

O estudo de estruturas de dados não pode ser desvinculado de seus aspectos algorítmicos. A escolha correta da estrutura adequada a cada caso depende diretamente do conhecimento de algoritmos para manipular a estrutura de maneira eficiente. O conhecimento de princípios de complexidade computacional é, portanto, requisito básico para se avaliar corretamente a adequação de uma estrutura de dados. Essa preocupação algorítmica constitui fator dominante na escolha da forma de apresentação das estruturas descritas neste texto.

Este capítulo contém, principalmente, a notação e os conceitos básicos utilizados ao longo do texto. A [Seção 1.2](#) descreve a linguagem em que os algoritmos serão apresentados. Alguns desses algoritmos são recursivos. Uma noção geral de recursividade é, então, apresentada na [Seção 1.3](#). A seção seguinte, 1.4, introduz o conceito de complexidade, utilizado para avaliar a eficiência de algoritmos. As complexidades são obtidas, geralmente, através de uma notação matemática especial, denominada notação *O*. Este é o objeto da [Seção 1.5](#). Finalmente, o conceito de algoritmo ótimo é descrito na [Seção 1.6](#).

1.2 Apresentação dos Algoritmos

Ao longo deste texto, os algoritmos serão descritos através de uma linguagem de leitura simples. Ela possui estrutura semelhante ao Pascal, por exemplo. Contudo, para facilitar a sua interpretação será adotado o estilo do livre formato, quando conveniente.

As convenções seguintes serão utilizadas com respeito à linguagem.

- A linguagem possui uma estrutura de blocos semelhante ao Pascal. O início e o final de cada bloco são determinados por endentação, isto é, pela posição da margem esquerda. Se uma certa linha do algoritmo inicia um bloco, ele se estende até a última linha seguinte, cuja margem esquerda se localiza mais à direita do que a primeira do bloco. Por exemplo, o bloco iniciado por **para** no [Algoritmo 1.1](#) inclui as três linhas seguintes.
- A declaração de atribuição é indicada pelo símbolo **:=**.
- As declarações seguintes são empregadas com significado semelhante ao usual.

se... então
 se... então... senão
 enquanto... faça
 para... faça
 pare

- Variáveis simples, vetores, matrizes e registros são considerados como tradicionalmente em linguagens de programação. Os elementos de vetores e matrizes são identificados por índices entre colchetes. Por exemplo, $A[5]$ e $B[i, 3]$ indicam, respectivamente, o quinto elemento do vetor A e o elemento identificado pelos índices $(i, 3)$ da matriz B . No caso de registros, a notação $T.chave$ indica o campo *chave* do registro T .
- A referência a registros pode ser também realizada por meio de ponteiros, que armazenam endereços, com o uso do símbolo $\uparrow$. Cada ponteiro é associado a um único tipo de registro. Por essa razão, o nome do registro pode ser omitido. Por exemplo, $pt \uparrow.info$ representa o campo *info* de um registro alocado no endereço contido em pt .
- São usados procedimentos e funções. A passagem de parâmetros é feita por referência, isto é, o endereço do parâmetro é transmitido para a rotina. Essa forma de transmissão possibilita a alteração do conteúdo da variável utilizada.
- A sentença imediatamente posterior ao símbolo % deve ser interpretada como comentário.

Como exemplo, seja uma sequência de elementos armazenada no vetor $S[i]$, $1 \leq i \leq n$. Deseja-se inverter os elementos da sequência no vetor, isto é, considerá-la de trás para a frente. Um algoritmo para resolver esse problema é simples. Basta trocar de posição o primeiro com o último elemento, em seguida o segundo com o antepenúltimo, e assim por diante. A formulação seguinte descreve o processo.

■ Algoritmo 1.1 Inversão de uma sequência

```
para  $i := 1, \dots, \lfloor n/2 \rfloor$  faça
   $temp := S[i]$ 
   $S[i] := S[n - i + 1]$ 
   $S[n - i + 1] := temp$ 
```

A notação $\lfloor x \rfloor$, que aparece no algoritmo, significa *piso* de x e representa o maior inteiro menor ou igual a x . Analogamente, $\lceil x \rceil$ é o *teto* de x e corresponde ao menor inteiro maior ou igual a x . Assim, $\lfloor 9/2 \rfloor = 4$ e $\lceil 9/2 \rceil = 5$.

1.3 Recursividade

Um tipo especial de procedimento será utilizado, algumas vezes, ao longo do texto. É aquele que contém, em sua descrição, uma ou mais chamadas a si mesmo. Um procedimento dessa natureza é denominado *recursivo*, e a chamada a si mesmo é dita *chamada recursiva*. Naturalmente, todo procedimento, recursivo ou não, deve possuir pelo menos uma chamada proveniente de um local exterior a ele. Essa chamada é denominada *externa*. Um procedimento não recursivo é, pois, aquele em que todas as chamadas são externas.

De modo geral, a todo procedimento recursivo corresponde um outro não recursivo que executa, exatamente, a mesma computação. Contudo, a recursividade pode apresentar vantagens concretas. Frequentemente, os procedimentos recursivos são mais concisos do que um não recursivo correspondente. Além disso, muitas vezes é aparente a relação direta entre um procedimento recursivo e uma prova por indução matemática. Nesses casos, a verificação da correção pode se tornar mais simples. Entretanto, muitas vezes há desvantagens no emprego prático da recursividade. Um algoritmo não recursivo equivalente pode ser mais eficiente.

O exemplo clássico mais simples de recursividade é o cálculo do fatorial de um inteiro $n \geq 0$. Um algoritmo recursivo para efetuar esse cálculo encontra-se descrito em seguida. A ideia do algoritmo é muito simples. Basta observar que o fatorial de n é n vezes o fatorial de $n - 1$, para $n > 0$. Por convenção, sabe-se que $0! = 1$. No algoritmo a seguir, as chamadas recursivas são representadas pela função *fat*. A chamada externa é *fat(n)*.

■ Algoritmo 1.2 Fatorial (recursivo)

função $fat(i)$

$fat(i) := \text{se } i \leq 1 \text{ então } 1 \text{ senão } i \times fat(i - 1)$

Para efeito de comparação, o [Algoritmo 1.3](#) descreve o cálculo do fatorial de n de forma não recursiva. A variável fat representa, agora, um vetor e não mais uma função. O elemento $fat[n]$ contém, no final, o valor do fatorial desejado.

■ Algoritmo 1.3 Fatorial (não recursivo)

$fat[0] := 1$

para $j := 1, \dots, n$ **faça**

$fat[j] := j \times fat[j - 1]$

Um exemplo conhecido, onde a solução recursiva é natural e intuitiva, é o do *Problema da Torre de Hanói*. Este consiste em três pinos, A , B e C , denominados *origem*, *destino* e *trabalho*, respectivamente, e n discos de diâmetros diferentes. Inicialmente, todos os discos se encontram empilhados no pino-origem, em ordem decrescente de tamanho, de baixo para cima. O objetivo é empilhar todos os discos no pino-destino, atendendo às seguintes restrições: (i) apenas um disco pode ser movido de cada vez, e (ii) qualquer disco não pode ser jamais colocado sobre outro de tamanho menor.

A solução do problema é descrita a seguir. Naturalmente, para $n > 1$, o pino-trabalho deve ser utilizado como área de armazenamento temporário. O raciocínio utilizado para resolver o problema é semelhante ao de uma prova matemática por indução. Suponha que se saiba como resolver o problema até $n - 1$ discos, $n > 1$, de forma recursiva. A extensão para n discos pode ser obtida pela realização dos seguintes passos:

- resolver o problema da Torre de Hanói para os $n - 1$ discos do topo do pino-origem A , supondo que o pino-destino seja C e o trabalho seja B ;
- mover o n -ésimo pino (maior de todos) de A para B ;
- resolver o problema da Torre de Hanói para os $n - 1$ discos localizados no pino C , suposto origem, considerando os pinos A e B como trabalho e destino, respectivamente.

Ao final desses passos, todos os discos se encontram empilhados no pino B e as duas restrições (i) e (ii) foram satisfeitas. O [Algoritmo 1.4](#) implementa o processo. O procedimento recursivo *hanoi* é utilizado com quatro parâmetros n , A , B e C , representando, respectivamente, o número de discos, o pino-origem, o destino e o trabalho.

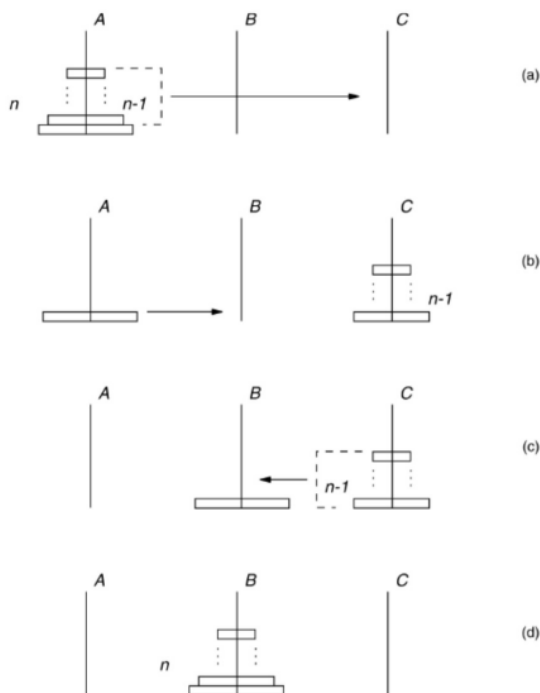


FIGURA 1.1 Problema da Torre de Hanói.

■ Algoritmo 1.4 Torre de Hanói

procedimento *hanoi*(n, A, B, C)

se $n > 0$ **então**

hanoi($n - 1, A, C, B$)

 mover o disco do topo de A para

B *hanoi*($n - 1, C, B, A$)

A [Figura 1.1](#) ilustra os passos efetuados pelo algoritmo. A chamada externa é *hanoi*(n, A, B, C).

1.4 Complexidade de Algoritmos

Conforme já mencionado, uma característica muito importante de qualquer algoritmo é o seu tempo de execução. Naturalmente, é possível determiná-lo através de métodos empíricos, isto é, obter o tempo de execução através da execução propriamente dita do algoritmo, considerando-se entradas diversas. Em contrapartida, é possível obter uma ordem de grandeza do tempo de execução através de métodos analíticos. O objetivo desses métodos é determinar uma expressão matemática que traduza o comportamento de tempo de um algoritmo. Ao contrário do método empírico, o analítico visa aferir o tempo de execução de forma independente do computador utilizado, da linguagem e dos compiladores empregados e das condições locais de processamento.

A tarefa de obter uma expressão matemática para avaliar o tempo de um algoritmo em geral não é simples, mesmo considerando-se uma expressão aproximada. As seguintes simplificações serão introduzidas para o modelo proposto.

- Suponha que a quantidade de dados a serem manipulados pelo algoritmo seja suficientemente grande. Isto é, algoritmos cujas entradas consistam em uma quantidade reduzida de dados não serão considerados. Somente o comportamento assintótico será avaliado, ou seja, a expressão matemática fornecerá valores de tempo que serão válidos unicamente quando a quantidade de dados correspondente crescer o suficiente.
- Não serão consideradas constantes aditivas ou multiplicativas na expressão matemática obtida. Isto é, a expressão matemática obtida será válida, a menos de tais constantes.

É necessário, ainda, definir a variável em relação à qual a expressão matemática avaliará o tempo de execução. O próprio conceito de algoritmo oferece a sugestão. Um algoritmo opera a partir de uma entrada para produzir uma saída, dentro de um tempo que se deseja avaliar. A ideia é exprimir o tempo de execução em função da entrada.

O processo de execução de um algoritmo pode ser dividido em etapas elementares, denominadas *passos*. Cada passo consiste na execução de um número fixo de operações básicas cujos tempos de execução são considerados constantes. A operação básica de maior frequência de execução no algoritmo é denominada *operação dominante*. Como a expressão do tempo de execução do algoritmo será obtida a menos de constantes aditivas e multiplicativas, o número de passos de um algoritmo pode ser interpretado como sendo o número de execuções da operação dominante. Por exemplo, em diversos algoritmos para ordenar os elementos de uma sequência dada, cada passo corresponde a uma comparação entre dois elementos da sequência. Na realidade, o número de passos de um algoritmo constitui a informação de que se necessita para avaliar o seu comportamento de tempo. Assim, um algoritmo de um único passo possui tempo de execução constante.

Pelo exposto, é natural definir a expressão matemática de avaliação do tempo de execução de um algoritmo como sendo uma função que fornece o número de passos efetuados pelo algoritmo a partir de uma certa entrada.

Como exemplo, no [Algoritmo 1.1](#) de inversão de sequência, cada entrada é uma sequência que se deseja inverter. O algoritmo efetua exatamente as mesmas operações para sequência de mesmo tamanho n . Cada passo corresponde à troca de posição entre dois elementos da sequência. Ou seja, à execução das três instruções de atribuição dentro do bloco **para** do algoritmo. O número de passos é, pois, igual ao número de execuções do bloco **para**. Isto é, igual a $\lfloor n/2 \rfloor$, $n > 1$.

Como exemplos adicionais, considere os problemas de determinar as matrizes soma C e produto D de duas matrizes dadas, $A = (a_{ij})$ e $B = (b_{ij})$, ambas $n \times n$. Nesse caso, C e D também possuem dimensão $n \times n$ e seus elementos c_{ij} e d_{ij} podem ser calculados, respectivamente, por

$$c_{ij} = a_{ij} + b_{ij}$$

$$d_{ij} = \sum_{1 \leq k \leq n} a_{ik} \cdot b_{kj}$$

O [Algoritmo 1.5](#) descreve a computação da matriz soma de duas matrizes, enquanto o [Algoritmo 1.6](#) fornece o produto.

■ **Algoritmo 1.5 Soma de matrizes**

```
para  $i := 1, \dots, n$  faça
  para  $j := 1, \dots, n$  faça
     $c_{ij} := a_{ij} + b_{ij}$ 
```

■ **Algoritmo 1.6 Produto de matrizes**

```
para  $i := 1, \dots, n$  faça
  para  $j := 1, \dots, n$  faça
     $c_{ij} := 0$ 
    para  $k := 1, \dots, n$  faça
       $c_{ij} := c_{ij} + a_{ik} \cdot b_{kj}$ 
```

Como no caso do [Algoritmo 1.1](#), ambos os algoritmos de soma e produto efetuam as mesmas operações, respectivamente, sempre que A, B forem matrizes de mesma dimensão $n \times n$. A variável independente é o parâmetro n . Cada passo do [Algoritmo 1.5](#) corresponde à execução de uma soma $a_{ij} + b_{ij}$, enquanto, no [Algoritmo 1.6](#), corresponde ao produto $a_{ik} \cdot b_{kj}$. O número total de passos é, pois, igual ao número total de somas $a_{ij} + b_{ij}$ e produtos $a_{ik} \cdot b_{kj}$, respectivamente, para cada caso. Ou seja, o [Algoritmo 1.5](#) efetua n^2 passos, enquanto o [Algoritmo 1.6](#) efetua n^3 .

A noção de complexidade de tempo é descrita a seguir.

Seja A um algoritmo, $\{E_1, \dots, E_m\}$, o conjunto de todas as entradas possíveis de A . Denote por t_i , o número de passos efetuados por A , quando a entrada for E_i . Definem-se

$$\text{complexidade do pior caso} = \max_{E_i \in E} \{t_i\},$$

$$\text{complexidade do melhor caso} = \min_{E_i \in E} \{t_i\},$$

$$\text{complexidade do caso médio} = \sum_{1 \leq i \leq m} p_i t_i,$$

onde p_i é a probabilidade de ocorrência da entrada E_i .

De forma análoga, podem ser definidas *complexidades de espaço* de um algoritmo.

As complexidades têm por objetivo avaliar a eficiência de tempo ou espaço. A complexidade de tempo de pior caso corresponde ao número de passos que o algoritmo efetua no seu pior caso de execução, isto é, para a entrada mais desfavorável. De certa forma, a complexidade de pior caso é a mais importante das três mencionadas. Ela fornece um limite superior para o número de passos que o algoritmo pode efetuar, em qualquer caso. Por isso mesmo é a mais utilizada. O termo *complexidade* será, então, empregado com o significado de complexidade de pior caso.

A complexidade de melhor caso é de uso bem menos frequente. É empregada em situações específicas. A complexidade de caso médio, apesar de importante, é menos utilizada do que a de pior caso por dois motivos. Em primeiro lugar, ela exige o prévio conhecimento da distribuição de probabilidades das diferentes entradas do algoritmo. Em diversos casos, essa distribuição é desconhecida. Além disso, o cálculo de seu valor $\sum p_i t_i$ frequentemente é de tratamento matemático complexo.

1.5 A Notação O

Observe que as definições de complexidade da seção anterior implicam o atendimento das duas simplificações mencionadas no início do parágrafo. Em consequência, quando se considera o número de passos efetuados por um algoritmo podem-se desprezar constantes aditivas ou multiplicativas. Por exemplo, um valor de número de passos igual a $3n$ será aproximado para n . Além disso, como o interesse é restrito a valores assintóticos, termos de menor grau também podem ser desprezados. Assim, um valor de número de passos igual a $n^2 + n$ será aproximado para n^2 . O valor $6n^3 + 4n - 9$ será transformado em n^3 .

Torna-se útil, portanto, descrever operadores matemáticos que sejam capazes de representar situações como essas. As notações O , Ω e Θ serão utilizadas com essa finalidade.

Sejam f, h funções reais positivas de variável inteira n . Diz-se que f é $O(h)$, escrevendo-se $f = O(h)$, quando existir uma constante $c > 0$ e um valor inteiro n_0 , tal que

$$n > n_0 \Rightarrow f(n) \leq c \cdot h(n).$$

Ou seja, a função h atua como um limite superior para valores assintóticos da função f . Em seguida são apresentados alguns exemplos da notação O .

$$f = n^2 - 1 \Rightarrow f = O(n^2).$$

$$f = n^2 - 1 \Rightarrow f = O(n^3).$$

$$f = 403 \Rightarrow f = O(1).$$

$$f = 5 + 2 \log n + 3 \log^2 n \Rightarrow f = O(\log^2 n).$$

$$f = 5 + 2 \log n + 3 \log^2 n \Rightarrow f = O(n).$$

$$f = 3n + 5 \log n + 2 \Rightarrow f = O(n).$$

$$f = 5 \cdot 2^n + 5n^{10} \Rightarrow f = O(2^n).$$

As seguintes propriedades são úteis para manipular expressões em notação O . Elas decorrem diretamente da definição.

Sejam g, h funções reais positivas e k uma constante. Então

(i) $O(g + h) = O(g) + O(h)$;

(ii) $O(k \cdot g) = k \cdot O(g) = O(g)$.

Essas duas propriedades foram empregadas, implicitamente, nos exemplos anteriores.

A notação O será utilizada, ao longo deste texto, para exprimir complexidades. Por exemplo, seja determinar as complexidades de pior, melhor e caso médio dos Algoritmos 1.1 a 1.6. Todos eles apresentam a propriedade de o número de passos manter-se o mesmo quando aplicados a entradas diferentes de mesmo tamanho. Ou seja, para um mesmo valor de n o número de passos mantém-se constante. Como a variável independente é o valor n , conclui-se que as complexidades de pior, melhor e caso médio são todas iguais entre si para cada algoritmo. O [Algoritmo 1.1](#) efetua sempre $\lfloor n/2 \rfloor$ passos. Logo, a sua complexidade é $O(n)$. No [Algoritmo 1.3](#), o número de passos é igual ao número de produtos $j \cdot fat(j - 1)$, isto é, n . Sua complexidade, portanto, é $O(n)$. Da mesma forma, verifica-se de imediato que as complexidades dos [Algoritmos 1.5](#) e [1.6](#) são iguais a $O(n^2)$ e $O(n^3)$, respectivamente.

Para encontrar a complexidade de procedimentos recursivos, pode-se aplicar a seguinte técnica. Determina-se o número total de chamadas ao procedimento recursivo. Em seguida, calcula-se a complexidade da execução correspondente a uma única chamada, sem que se considerem as chamadas recursivas encontradas. A complexidade total será o produto do número de chamadas pela complexidade da computação de uma chamada isolada. Por exemplo, para calcular o fatorial de $n > 0$, de forma recursiva, o [Algoritmo 1.2](#) efetua um total de n chamadas ao procedimento *fat*. A complexidade da computação correspondente a uma chamada é constante, isto é, $O(1)$. De fato, para $n > 1$, apenas um produto é efetuado e, quando $n \leq 1$, apenas uma atribuição é executada. Logo, a complexidade final do algoritmo é $O(n)$. A complexidade do algoritmo recursivo 1.4, para resolver o problema da Torre de Hanói, é $O(2^n)$, segundo o [Exercício 1.4](#).

Conforme mencionado, os algoritmos até agora examinados efetuam exatamente os mesmos passos para entradas diferentes de mesmo tamanho. Sem dúvida, este não é o caso geral. Como exemplo, suponha um problema cuja entrada consista em duas matrizes A e B , de dimensões $n \times n$, e um parâmetro binário x , com valores possíveis 0 e 1. Dependendo do valor de x , deve-se calcular a soma ou o produto das matrizes. Isto é, se $x = 0$, calcular a matriz $A + B$, ou, se $x = 1$, calcular $A \cdot B$. Um algoritmo para efetuar essa tarefa é simples. Sua entrada consiste em $n^2 + 1$ informações, isto é, possui tamanho $O(n^2)$. Em seguida, verifica-se o valor x obtido da entrada. Se $x = 0$, então executa-se o [Algoritmo 1.5](#) da soma de matrizes. Se $x = 1$, o [Algoritmo 1.6](#) de produto de matrizes. Nesse caso, para cada valor de n , o algoritmo é sensível a duas entradas distintas, correspondendo aos valores 0 e 1 de x , respectivamente. A complexidade de melhor caso corresponde a $x = 0$ e é dada pela complexidade do [Algoritmo 1.5](#), ou seja, $O(n^2)$. O pior caso ocorre quando $x = 1$. Sua complexidade é obtida do [Algoritmo 1.6](#), isto é, $O(n^3)$. Para determinar a complexidade do caso médio, seja q a probabilidade de que o valor de x , da entrada, seja igual a 0. Então, a expressão da

complexidade do caso médio é $q n^2 + (1 - q)n^3$. Um outro exemplo de complexidade de caso médio será descrito no próximo capítulo.

A notação Θ , descrita a seguir, é útil para exprimir limites superiores justos. Sejam f, g funções reais positivas da variável inteira n . Diz-se que f é $\Theta(g)$, escrevendo-se $f = \Theta(g)$, quando ambas as condições $f = O(g)$ e $g = O(f)$ forem verificadas. A notação Θ exprime o fato de que duas funções possuem a mesma ordem de grandeza assintótica. Por exemplo, se $f = n^2 - 1$, $g = n^2$ e $h = n^3$, então f é $O(g)$, f é $O(h)$, g é $O(f)$, mas h não é $O(f)$. Consequentemente, f é $\Theta(g)$, mas f não é $\Theta(h)$. Da mesma forma, se $f = 5 + 2 \log n + \log^2 n$ e $g = n$, então f é $O(g)$, porém f não é $\Theta(g)$. No caso, f é $\Theta(\log^2 n)$.

Assim como a notação O é útil para descrever limites superiores assintóticos, a notação Ω , definida a seguir, é empregada para limites inferiores assintóticos.

Sejam f, h funções reais positivas da variável inteira n . Diz-se que f é $\Omega(h)$, escrevendo-se $f = \Omega(h)$ quando existir uma constante $c > 0$ e um valor inteiro n_0 , tal que

$$n > n_0 \Rightarrow f(n) \geq c \cdot h(n).$$

Por exemplo, se $f = n^2 - 1$, então são válidas as igualdades $f = \Omega(n^2)$, $f = \Omega(n)$ e $f = \Omega(1)$, mas não vale $f = \Omega(n^3)$.

1.6 Algoritmos Ótimos

A noção de complexidade está relacionada a um dado algoritmo. Ela visa determinar o número de passos efetuados por um algoritmo específico, sem levar em consideração a possível existência de outros algoritmos para o mesmo problema. Essa questão mais abrangente será abordada na presente seção.

Seja P um problema. Um *limite inferior para P* é uma função ℓ , tal que a complexidade de pior caso de qualquer algoritmo que resolva P é $\Omega(\ell)$. Isto é, todo algoritmo que resolve P efetua, pelo menos, $\Omega(\ell)$ passos. Se existir um algoritmo A , cuja complexidade seja $O(\ell)$, então A é denominado *algoritmo ótimo* para P . Nesse caso, o limite $\Omega(\ell)$ é o melhor (maior) possível.

Intuitivamente, um algoritmo ótimo é aquele que apresenta a menor complexidade dentre todos os possíveis algoritmos para resolver o mesmo problema. Assim como a notação O é conveniente para exprimir complexidades, a notação Ω é utilizada para limites inferiores.

Existem limites inferiores naturais, como, por exemplo, o tamanho da entrada. Todo possível algoritmo para o problema considerado deverá, necessariamente, efetuar a leitura da entrada. Assim, por exemplo, a entrada do [Algoritmo 1.1](#), de inversão de sequência, consiste em uma sequência de n elementos. Qualquer possível algoritmo para inverter a sequência deverá efetuar a sua leitura. Isto é, um limite inferior para o problema de inversão de sequência é $\Omega(n)$. A complexidade do [Algoritmo 1.1](#) é $O(n)$. Conclui-se, então, que ele é um algoritmo ótimo. Da mesma forma, qualquer algoritmo para somar duas matrizes $n \times n$ deverá, em primeiro lugar, efetuar a sua leitura. Isto é, um limite inferior para o problema de soma de matrizes é $\Omega(n^2)$. A complexidade do [Algoritmo 1.5](#), para somar as matrizes, é $O(n^2)$. Logo, ele é ótimo. Analogamente, um limite inferior para o problema do produto de matrizes é $\Omega(n^2)$. A complexidade do [Algoritmo 1.6](#), que calcula o produto de matrizes, é $O(n^3)$. Em princípio, nada poderia ser dito acerca desse algoritmo, quanto a reconhecê-lo como ótimo ou não. Contudo, são conhecidos algoritmos que efetuam o produto de matrizes dentro de uma complexidade inferior a $O(n^3)$. Isso significa que o [Algoritmo 1.6](#) não é ótimo.

De modo geral, o interesse é determinar a função que represente o *maior* limite inferior possível para um problema. Analogamente, para um certo algoritmo o interesse é encontrar a função representativa da *menor* complexidade de pior caso do algoritmo. A determinação de complexidades justas é realizada, sem dificuldades, para uma grande quantidade de algoritmos conhecidos. O mesmo não se dá, contudo, para os limites inferiores. Sem dúvida, a determinação de limites inferiores triviais, como os provenientes do tamanho da entrada, pode ser de pouco interesse se os valores obtidos forem de grandeza sensivelmente inferior às complexidades dos algoritmos conhecidos. O cálculo de limites inferiores, de modo geral, não é um problema simples. Esse cálculo se baseia no desenvolvimento de propriedades matemáticas do problema, independentemente dos algoritmos empregados. Na realidade, são relativamente poucos os problemas para os quais existem limites inferiores não triviais conhecidos, como, por exemplo, o problema de ordenar uma sequência de n elementos. O limite trivial, extraído da leitura, é $\Omega(n)$. Contudo,

há uma prova matemática de que $\Omega(n \log n)$ é um limite inferior. Por outro lado, existem algoritmos conhecidos de ordenação cujas complexidades são $O(n \log n)$. Isso permite concluir que tais algoritmos são ótimos e que o limite $\Omega(n \log n)$ é o melhor possível. Para uma quantidade imensa de problemas de interesse, a distância entre o melhor (maior) limite inferior conhecido e o algoritmo de melhor (menor) complexidade é grande. Por exemplo, existem muitos problemas cujo maior limite inferior conhecido é um polinômio na variável representativa do tamanho da entrada, enquanto a menor complexidade de um algoritmo conhecido é exponencial nesta variável. Em todos esses casos, permanece a dúvida quanto a se é possível provar um novo limite inferior mais alto ou desenvolver um novo algoritmo de complexidade mais baixa, ou ambos.

1.7 Exercícios

1.1 Responder se é certo ou errado:

Todo procedimento recursivo deve incorporar terminações sem chamadas recursivas, caso contrário ele seria executado um número infinito de vezes.

1.2 Responder se é certa ou errada cada afirmativa abaixo:

- (i) O [Algoritmo 1.2](#), que calcula o fatorial de forma recursiva, requer apenas uma quantidade constante de memória.
- (ii) O [Algoritmo 1.3](#), fatorial não recursivo, requer o armazenamento do vetor *fat*, com $n + 1$ elementos.

1.3 Desenvolver um algoritmo não recursivo para o cálculo do fatorial de inteiro $n \geq 0$, de tal forma que prescindia do armazenamento de qualquer vetor.

1.4 Mostrar que o [Algoritmo 1.4](#), para o problema da Torre de Hanói, requer exatamente $2^n - 1$ movimentos de disco para terminar.

◦ 1.5 Provar que é mínimo o número de movimentos de discos no problema da Torre de Hanói, dado no exercício anterior.

1.6 Escrever uma prova de correção para o [Algoritmo 1.4](#).

1.7 Reescrever o [Algoritmo 1.4](#), de forma que a recursividade pare no nível correspondente a $n = 1$, e não a $n = 0$, como no algoritmo do texto. Há alguma vantagem em realizar essa modificação? Qual?

◦ 1.8 Elaborar um algoritmo não recursivo para o problema da Torre de Hanói.

1.9 Considere a seguinte generalização do problema da Torre de Hanói. O problema, agora, consiste em n discos de tamanhos distintos e quatro pinos, respectivamente, o de origem, o de destino e dois pinos de trabalho. De resto, o problema é como no caso de três pinos. Isto é, de início, os discos se encontram todos no pino-origem, em ordem decrescente de tamanho, de baixo para cima. O objetivo é empilhar todos os discos no pino-destino, satisfazendo às condições (i) e (ii) do caso dos três pinos, descrito na [Seção 1.3](#).

Elaborar um algoritmo para resolver essa generalização. Determinar o número de movimentos de disco efetuados.

• 1.10 Elaborar um algoritmo para resolver o problema da Torre de Hanói, com quatro pinos, cujo número de movimentos de disco seja mínimo.

◦ 1.11 Elaborar um algoritmo para resolver o problema da Torre de Hanói com $m \geq 3$ pinos, isto é, em que $m - 2$ pinos são de trabalho. Determinar o número de movimentos de disco efetuados.

◦ 1.12 Provar ou dar contraexemplo:

A solução do problema da Torre de Hanói é única. Isto é, só há uma única sequência de movimentos de discos que conduz à solução, a menos de repetições de movimentos.

1.13 Repetir o [Exercício 1.12](#) para o caso de quatro pinos.

1.14 Escrever as seguintes funções em notação O :

$$n^3 - 1; n^2 + 2 \log n; 3n^n + 5 \cdot 2^n; (n-1)^n + n^{n-1}; 302.$$

1.15 Responder se é certo ou errado:

Se f, g são funções tais que $f = O(g)$ e $g = \Omega(f)$, então $f = \Theta(g)$.

1.16 Responder se é certo ou errado:

A definição da notação Θ , dada na [Seção 1.5](#), é equivalente à seguinte: sejam f, h funções reais positivas da variável inteira n . Diz-se que $f = \Theta(h)$ quando existirem constantes $c, d > 0$ e um valor inteiro n_0 , tal que

$$n > n_0 \Rightarrow c \cdot h(n) \leq f(n) \leq d \cdot h(n).$$